Credit Risk Modeling on AWS

Batch Scoring & Credit Limit Management

Batch Scoring & Limit Management Architecture

What This Guide Covers:

- EventBridge scheduled execution of limit management process
- Batch credit scoring with Amazon Redshift
- SageMaker Pipelines for ML workflow orchestration
- Business rules for credit limit management with Lambda function
- How to store the limit change decisions into data warehouse for reporting and analytics

January 2026

Overview

This chapter demonstrates a batch credit scoring and limit increase system using SageMaker Pipelines for ML workflow orchestration, EventBridge for scheduled execution, and Amazon Redshift (or PostgreSQL locally) for storing customer batch scores. The architecture processes existing customers in batches to evaluate credit limit increase eligibility. For local development, we use PostgreSQL (compatible with Redshift's PostgreSQL protocol on port 5439) and SageMaker Pipelines in local mode.

Loan limit management for revolving credit products (lines of credit, credit cards, overdrafts, etc.) is essential for credit risk management. It ensures banks don't overlend to customers who may draw down their limit prior to default, while avoiding unnecessary restrictions on creditworthy borrowers. Limit management relies on behavioral scoring rather than application scoring used when customers first apply. Behavioral scoring incorporates internal data collected after the initial loan decision and typically has higher predictive power.

A common practice is to issue small limits at the start of the relationship and increase them gradually as the customer builds credit history with the bank. Customers who immediately max out their limit are more likely to default. A key risk driver in behavioral scoring is credit utilization: at 80-90% utilization, default risk increases significantly. Other drivers include account balance volatility, negative bureau flags, and days with negative balances. In practice, limit increases are more common than decreases, as reducing limits can harm customer relationships and attract regulatory scrutiny.

Design Rationale: Separation of Concerns

This architecture separates model scoring from business rule application, reflecting real-world organizational structures:

- **Data Science Team**: Focuses on model development and scoring accuracy. The pipeline handles training, data querying, and inference, delivering credit scores as outputs.
- **Credit Risk Team**: Owns business rules and limit management decisions. The Lambda function applies these rules independently, allowing frequent policy updates without modifying the ML pipeline.

This decoupling provides several benefits:

- **Agility**: Business rules can change frequently (regulatory updates, risk appetite adjustments) without requiring ML pipeline modifications or redeployments
- **Maintainability**: Data scientists focus on model performance; risk analysts focus on policy logic
- **Governance**: Clear separation of responsibilities aligns with typical organizational structures
- **Testing**: Business rules can be tested independently of model training and inference

While it's technically possible to embed business rules within the pipeline, this separation provides better flexibility and aligns with common industry practices where scoring and decision-making are distinct responsibilities.

Key Learning Objectives

By the end of this guide, you will understand:

- Scheduled limit management process with EventBridge (CloudWatch Events) triggering Lambda functions
- How to orchestrate multi-step ML workflows with SageMaker Pipelines (local mode)
- How to integrate Lambda functions with SageMaker Pipelines for business logic
- How to implement business rules for credit limit management (INCREASE/DECREASE/-KEEP) with Lambda function
- How to store the limit change decisions into data warehouse for reporting and analytics

Architecture Components

The batch scoring system consists of a workflow orchestrated by SageMaker Pipelines with three steps: **TrainCatBoostModel** (Training Step), **QueryBatchScores** (Processing Step), and **SageMakerInference** (Processing Step), followed by a Lambda function that applies business rules (**LimitManagerLambda**) and updates the database. In production, the training step would typically run periodically (e.g., monthly) while scoring runs daily. The workflow can be triggered by EventBridge on a cron schedule (e.g., daily at 2 AM UTC) or executed on demand.

Workflow Overview

Batch Scoring & Limit Management Pipeline

The workflow is executed via SageMaker Pipelines (can be triggered by EventBridge on schedule) and executes the following steps:

1. **EventBridge**: Can trigger Pipeline executions on schedule (daily at 2 AM UTC) via Lambda function or on demand
2. **SageMaker Pipelines**: Orchestrates the three-step ML workflow (Training + 2 Processing Steps)
3. **Train CatBoost Model** (Training Step): Trains credit scoring model with SHAP values and PDO scoring
4. **Query Batch Scores** (Processing Step): Queries PostgreSQL/Redshift for eligible customers
5. **SageMaker Inference** (Processing Step): Gets updated credit scores using trained CatBoost model
6. **Lambda Function**: Applies business rules (INCREASE/DECREASE/KEEP) to determine limit changes
7. **Update Database**: Updates PostgreSQL/Redshift with limit change decisions

SageMaker Pipelines Workflow

The pipeline defines a linear workflow with three steps (one Training Step and two Processing Steps) that execute sequentially. After the pipeline completes, a Lambda function is invoked to apply business rules and update the database:

Pipeline Execution Flow

```
TrainCatBoostModel (Step 0)
    | [model.tar.gz -> S3]
    v
QueryBatchScores (Step 1)
    | [customers.json -> S3]
    v
SageMakerInference (Step 2)
    | [scored_customers.json -> S3]
    v
[Pipeline Complete]
    |
    v
Lambda Function (LimitManagerLambda)
    | [Reads scored_customers.json from S3]
    | [Applies business rules: INCREASE/DECREASE/KEEP]
    | [decisions.json -> S3]
    v
Update Database (PostgreSQL/Redshift)
    | [Database updated with decisions]
    v
Complete
```

Detailed Step Descriptions

1. **TrainCatBoostModel** (Training Step): Trains CatBoost credit scoring model
   - Uses training data from `data/` directory
   - Trains model with SHAP values and PDO scoring
   - Saves model artifacts to S3 (`model.tar.gz`)
   - Model includes `catboost_model.joblib` and `model_metadata.json`
2. **QueryBatchScores** (Processing Step): Queries PostgreSQL/Redshift for customers with:
   - Current score $\geq$ 600 (good credit)
   - Current limit $<$ \$10,000
   - No previous limit increase decision

   Outputs eligible customers to S3 for next step.
3. **SageMakerInference** (Processing Step): Runs CatBoost model inference to score eligible customers
   - Loads trained model from training step output (S3)
   - Extracts model from `model.tar.gz` automatically
   - Uses SHAP values for feature importance

- Converts log-odds to scores using PDO method

Outputs scored customers to S3.

4. **Lambda Function (LimitManagerLambda)**: Applies business rules to determine limit changes
   - Reads `scored_customers.json` from S3 (output from inference step)
   - Applies business rules: INCREASE, DECREASE, or KEEP
   - Saves decisions to S3 as `decisions.json`
   - Returns S3 URI of decisions for database update
5. **Update Database**: Updates PostgreSQL/Redshift with decisions and new limits
   - Reads `decisions.json` from S3
   - Connects to database (PostgreSQL locally, Redshift in production)
   - Updates customer records with decisions and new limits

Benefits of SageMaker Pipelines

- **ML-native**: Built specifically for ML workflows
- **Versioning**: All pipeline executions are tracked and versioned
- **Reproducibility**: Inputs and outputs are logged for each run
- **Local mode**: Can run locally for development and testing
- **Monitoring**: Built-in monitoring and logging capabilities

CatBoost Model with SHAP Values and PDO Scoring

CatBoost + SHAP + PDO Method

This chapter uses CatBoost for credit scoring with SHAP values converted to scores using the PDO (Points to Double Odds) method.

Model Training:

- **Algorithm**: CatBoost (Gradient Boosting with categorical handling)
- **SHAP Values**: Feature importance using `ShapValues`
- **Scoring**: Convert SHAP values to credit scores using PDO formula

PDO Scorecard Formula:

$$\text{Score} = \text{offset} + \text{factor} \times (-\text{log\_odds})$$

Where:

- $\text{factor} = \frac{\text{pts\_double\_odds}}{\ln(2)} = \frac{20}{\ln(2)} \approx 28.85$
- $\text{offset} = \text{target\_score} - \text{factor} \times \ln(\text{target\_odds}) = 600 - 28.85 \times \ln(30) \approx 501.86$ (actual: 501.86)
- $\text{log\_odds} = \sum \text{SHAP\_values}$ (sum of all SHAP feature contributions + base value)
- Target score: 600, Target odds: 30, Points to double odds: 20

Why PDO?

- Standard credit scoring convention (same as Chapter 1)
- Interpretable scores in 300-850 range
- Consistent scaling across different models
- Negative sign ensures higher scores = better credit (lower default risk)

SHAP Values to Points Conversion

The conversion from SHAP values to credit scores follows these steps:

Step 1: Calculate SHAP Values

- Use CatBoost's `get_feature_importance(type='ShapValues')` method
- Returns matrix of shape $(n\_samples, n\_features + 1)$
- Last column: base value (expected log-odds)
- First $n\_features$ columns: feature contributions (in log-odds space)

Step 2: Sum SHAP Contributions

$$\text{log\_odds} = \sum_{i=1}^{n} \text{SHAP}_i + \text{base\_value}$$

Where each SHAP_i represents the contribution of feature i to the log-odds of default.

Step 3: Convert to Credit Score using PDO

$$\text{Score} = \text{offset} + \text{factor} \times (-\text{log\_odds})$$

The negative sign ensures that:

- Higher log-odds (higher default risk) $\rightarrow$ Lower score
- Lower log-odds (lower default risk) $\rightarrow$ Higher score
- Consistent with credit scoring convention (higher = better)

Example Calculation

- Feature SHAP contributions: $[0.5, -0.3, 0.2, -0.1, 0.4, -0.2, 0.1, -0.3]$
- Base value: -2.5
- Total log-odds: $0.5 - 0.3 + 0.2 - 0.1 + 0.4 - 0.2 + 0.1 - 0.3 - 2.5 = -2.2$
- Score: $501.86 + 28.85 \times (-(-2.2)) = 501.86 + 63.47 = 565.33$

Note on WOE and PDO Sign Convention

In conventional credit scoring, WOE (Weight of Evidence) is often calculated as:

$$\text{WOE}_i = \ln\left(\frac{\%\text{ of Goods}_i}{\%\text{ of Bads}_i}\right)$$

This formulation reverses the sign compared to log-odds, which is linked to scores having higher values for good risk and lower values for bad risk. In our implementation, we manage this convention by reversing the sign of log odds in the PDO score formula, ensuring consistency with standard credit scoring practices where higher scores indicate better creditworthiness.

Optimal Cutoff Determination

The cutoff is determined by maximizing the Kolmogorov-Smirnov (KS) statistic, which measures the separation between good and bad credit risk:

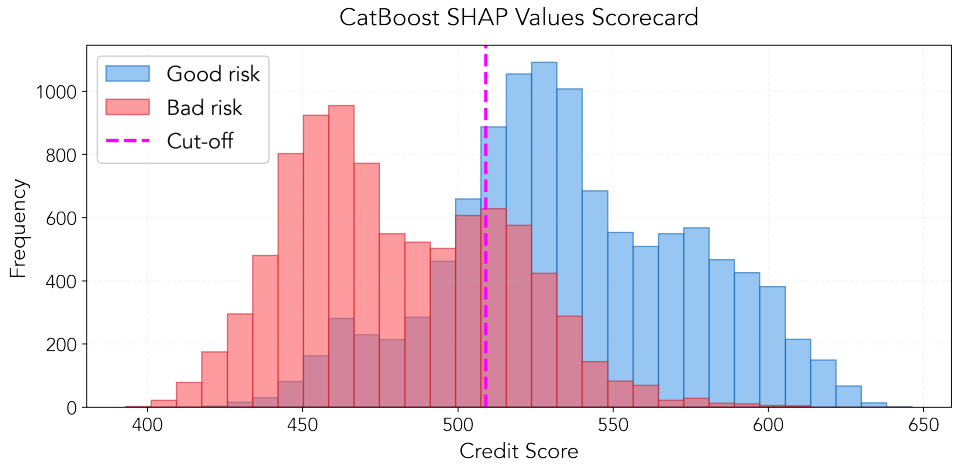
$$\text{KS} = \max_{\text{threshold}} \left|\text{CDF}_{\text{bads}}(\text{threshold}) - \text{CDF}_{\text{goods}}(\text{threshold})\right|$$

Where:

- CDF_{bads}: Cumulative distribution function of bad risk (defaults)
- $\text{CDF}_{\text{goods}}$: Cumulative distribution function of good risk (non-defaults)
- The threshold that maximizes KS provides optimal separation

For our trained CatBoost model:

- **Optimal cutoff**: 563 points (maximizes KS statistic)
- **Gini coefficient**: 0.8640 (excellent discriminatory power)
- **PDO parameters**: factor=28.85, offset=501.86



The histogram shows separation between good (blue) and bad (red) credit risk using CatBoost SHAP values converted to scores via PDO. The fuchsia dashed line indicates the optimal cutoff at 563 points.

Business Rules for Limit Changes

Limit Change Decisions

The Lambda function applies the following business rules to determine credit limit changes (INCREASE, DECREASE, or KEEP):

Decision Logic:

$$\text{Decision} = \begin{cases} \text{INCREASE} & \text{if New Score} \geq 700 \text{ and Current Limit} < \$10,000 \\ \text{INCREASE} & \text{if New Score} \geq 600 \text{ and Current Limit} < \$5,000 \\ \text{DECREASE} & \text{if New Score} < 500 \\ \text{KEEP} & \text{otherwise} \end{cases}$$

Increase Calculation:

$$\text{New Limit} = \begin{cases} \text{Current Limit} \times (1 + \min(50\%, \frac{\text{New Score} - 600}{2}\%)) & \text{if New Score} \geq 700 \\ \text{Current Limit} \times (1 + \min(25\%, \frac{\text{New Score} - 550}{2}\%)) & \text{if New Score} \geq 600 \\ \min(\text{Calculated Limit}, \$10,000) & \text{Cap at maximum} \end{cases}$$

Decrease Calculation:

$$\text{New Limit} = \begin{cases} \text{Current Limit} \times (1 - \max(10\%, \frac{500 - \text{New Score}}{10}\%)) & \text{if New Score} < 500 \\ \max(\text{Calculated Limit}, \$1,000) & \text{Floor at minimum} \end{cases}$$

Examples:

- Customer with score 720 and limit $4,000: New limit = $6,000 (50% increase, capped at $10,000)
- Customer with score 650 and limit $3,000: New limit = $3,750 (25% increase, capped at $5,000)
- Customer with score 450 and limit $5,000: New limit = $4,500 (10% decrease, minimum $1,000)
- Customer with score 550: KEEP (no change)

Scheduled Execution with EventBridge

EventBridge Scheduling

EventBridge (CloudWatch Events) triggers SageMaker Pipelines executions on a schedule via a Lambda function (`PipelineTriggerLambda`):

Schedule Expression: `cron(0 2 * * ? *)`

- Default: Runs daily at 2:00 AM UTC
- Customizable to any schedule (hourly, weekly, etc.) by modifying `SCHEDULE_EXPRESSION` in `setup_eventbridge.py`
- Triggers `PipelineTriggerLambda` which starts the SageMaker Pipeline

Complete Architecture Flow:

1. **EventBridge Rule** (`batch-scoring-schedule`): Scheduled trigger (daily at 2 AM UTC)
2. **PipelineTriggerLambda**: Invoked by EventBridge, starts the SageMaker Pipeline
3. **SageMaker Pipelines**: Executes TrainCatBoostModel, QueryBatchScores, SageMaker-Inference steps
4. **LimitManagerLambda**: Applies business rules (INCREASE/DECREASE/KEEP) after pipeline completes
5. **Database Update**: Updates PostgreSQL/Redshift with decisions

Note: In LocalStack, EventBridge triggers a Lambda function that starts the pipeline. On real AWS, EventBridge can directly trigger SageMaker Pipelines, but the Lambda approach works in both environments and is included in the standard workflow (`make run-workflow`).

SageMaker Pipelines Architecture

Pipeline Structure and Data Flow

Pipeline: `BatchCreditScoringPipeline` - Linear workflow with 3 steps (1 Training + 2 Processing), followed by Lambda and database update

Data Flow:

- Step 0: Trains model and saves to S3 (`model.tar.gz`)
- Step 0 to Step 2: Model artifacts to inference step
- Step 1 to Step 2: `customers.json` to S3
- Step 2: `scored_customers.json` to S3 (pipeline output)
- Lambda: Reads `scored_customers.json` from S3, applies business rules, saves `decisions.json` to S3
- Database Update: Reads `decisions.json` from S3, updates PostgreSQL/Redshift

Configuration: Local mode, python:3.11-slim containers, sequential execution, Lambda invoked after pipeline completion

Code Examples

Complete Pipeline Implementation

This is the complete pipeline implementation showing all steps and their connections:

```
import json
import os
import subprocess
from pathlib import Path
import boto3
from sagemaker.estimator import Estimator
from sagemaker.local import LocalSession
from sagemaker.processing import (
   ProcessingInput, ProcessingOutput, ScriptProcessor
)
from sagemaker.workflow.pipeline import Pipeline
from sagemaker.workflow.steps import ProcessingStep, TrainingStep

# Configuration
ENDPOINT_URL = "http://localhost:4566"
S3_BUCKET = "credit-scoring-models"
REGION = "us-east-1"
DUMMY_IAM_ROLE = (
   "arn:aws:iam::111111111111:role/service-role/"
   "AmazonSageMaker-ExecutionRole-20200101T000001"
)

# Configure LocalStack S3
os.environ["AWS_ENDPOINT_URL_S3"] = ENDPOINT_URL
os.environ["AWS_ACCESS_KEY_ID"] = "test"
os.environ["AWS_SECRET_ACCESS_KEY"] = "test"

boto3_session = boto3.Session(
   aws_access_key_id="test",
   aws_secret_access_key="test",
   region_name=REGION,
)

s3_client = boto3_session.client("s3", endpoint_url=ENDPOINT_URL) # LocalStack S3
    client
sagemaker_session = LocalSession(boto_session=boto3_session)
sagemaker_session.config = {"local": {"local_code": True}} # SageMaker Local Mode
sagemaker_session.default_bucket = lambda: S3_BUCKET

# Create shared processor
def create_processor():
   return ScriptProcessor(
      role=DUMMY_IAM_ROLE,
      image_uri="python:3.11-slim",
      command=["python"],
      instance_type="local",
      instance_count=1,
      sagemaker_session=sagemaker_session,
      base_job_name="credit-scoring",
   )

# Step 0: Train CatBoost model
def create_training_step():
```

```
    data_dir = "data"
    estimator = Estimator(
        image_uri="catboost-sagemaker:latest",
        role=DUMMY_IAM_ROLE,
        instance_type="local",
        instance_count=1,
        sagemaker_session=sagemaker_session,
        output_path=f"s3://{S3_BUCKET}/models",
        hyperparameters={
            "iterations": 100,
            "learning-rate": 0.1,
            "depth": 6,
            "target-score": 600,
            "target-odds": 30,
            "pts-double-odds": 20,
        },
        entry_point="training/train.py",
    )
    return TrainingStep(
        name="TrainCatBoostModel",
        estimator=estimator,
        inputs={"train": f"file://{data_dir}"},
    )

# Step 1: Query eligible customers
def create_query_step(processor):
    script_path = "scripts/database.py"
    return ProcessingStep(
        name="QueryBatchScores",
        processor=processor,
        outputs=[
            ProcessingOutput(
                output_name="customers",
                source="/opt/ml/processing/output",
                destination=f"s3://{S3_BUCKET}/pipeline/customers",
            )
        ],
        code=script_path,
        job_arguments=[
            "query", # Command to execute
            "--db-host", "host.docker.internal",
            "--db-port", "5432",
            "--db-name", "credit_scoring",
            "--db-user", "creditrisk",
            "--db-password", "creditrisk123",
            "--limit", "20",
            "--output-path", "/opt/ml/processing/output/customers.json",
        ],
    )

# Step 2: Run inference
def create_inference_step(processor, query_step, training_step):
    script_path = "scripts/inference_processing.py"
    return ProcessingStep(
        name="SageMakerInference",
        processor=processor,
        inputs=[
            ProcessingInput(
                source=query_step.properties.ProcessingOutputConfig
                    .Outputs["customers"].S3Output.S3Uri,
                destination="/opt/ml/processing/input/customers",
```

```
            input_name="customers",
         ),
         ProcessingInput(
            source=training_step.properties.ModelArtifacts.S3ModelArtifacts,
            destination="/opt/ml/model",
            input_name="model",
         ),
      ],
      outputs=[
         ProcessingOutput(
            output_name="scored_customers",
            source="/opt/ml/processing/output",
            destination=f"s3://{S3_BUCKET}/pipeline/scored_customers",
         )
      ],
      code=script_path,
      job_arguments=["--model-dir", "/opt/ml/model"],
      depends_on=[query_step, training_step],
   )

# Invoke Lambda function after pipeline completes
def invoke_lambda_function(s3_uri: str) -> str:
   """Invoke Lambda function to make decisions."""
   function_name = "LimitManagerLambda"
   lambda_client = boto3.client("lambda", endpoint_url=ENDPOINT_URL)

   # Invoke Lambda function
   response = lambda_client.invoke(
      FunctionName=function_name,
      InvocationType="RequestResponse",
      Payload=json.dumps({"s3_uri": s3_uri}),
   )

   # Parse response
   result = json.loads(response["Payload"].read().decode("utf-8"))
   return result.get("s3_uri")

# Update database from S3
def update_database_from_s3(decisions_s3_uri: str):
   """Update PostgreSQL database with decisions from S3."""
   # Parse S3 URI
   decisions_s3_uri = decisions_s3_uri.removeprefix("s3://")
   parts = decisions_s3_uri.split("/", 1)
   bucket = parts[0]
   key = parts[1] if len(parts) > 1 else ""

   # Download decisions file from S3
   decisions_file = Path("temp_decisions.json")
   response = s3_client.get_object(Bucket=bucket, Key=key)
   content = response["Body"].read().decode("utf-8")
   with open(decisions_file, "w") as f:
      f.write(content)

   # Run database update script
   subprocess.run([
      "python", "scripts/database.py", "update",
      "--decisions-file", str(decisions_file),
      "--db-host", "localhost",
      "--db-port", "5432",
      "--db-name", "credit_scoring",
      "--db-user", "creditrisk",
```

```
        "--db-password", "creditrisk123",
    ], check=True)

    # Clean up temp file
    if decisions_file.exists():
        decisions_file.unlink()

# Create complete pipeline
def create_pipeline():
    processor = create_processor()
    training_step = create_training_step()
    query_step = create_query_step(processor)
    inference_step = create_inference_step(processor, query_step, training_step)

    return Pipeline(
        name="BatchCreditScoringPipeline",
        steps=[training_step, query_step, inference_step],
        sagemaker_session=sagemaker_session,
    )

# Execute pipeline workflow
def execute_pipeline_workflow():
    # Setup S3 bucket
    try:
        s3_client.head_bucket(Bucket=S3_BUCKET)
    except Exception:
        s3_client.create_bucket(Bucket=S3_BUCKET)

    # Create and register pipeline
    pipeline = create_pipeline()
    pipeline.upsert(role_arn=DUMMY_IAM_ROLE)

    # Execute pipeline
    execution = pipeline.start()
    try:
        execution.wait()
    except AttributeError:
        # Local mode - execution completes synchronously
        pass

    # After pipeline completes, invoke Lambda
    scored_customers_uri = f"s3://{S3_BUCKET}/pipeline/scored_customers/scored_customers
        .json"
    decisions_uri = invoke_lambda_function(scored_customers_uri)

    # Update database
    update_database_from_s3(decisions_uri)
    print("Workflow completed successfully")

# Execute pipeline
if __name__ == "__main__":
    execute_pipeline_workflow()
```

Query Batch Scores Processing Step

This Processing Step (runs in a SageMaker Processing container) queries Amazon Redshift (or PostgreSQL locally) for customers eligible for limit increase evaluation. The script is invoked with CLI arguments from the pipeline step.

```python
# scripts/database.py - query command
import json
import psycopg2

def query_eligible_customers(db_host, db_port, db_name, db_user, db_password,
                      limit=20, output_path=None):
   """Query eligible customers for batch scoring."""
   conn = psycopg2.connect(
      host=db_host,
      port=db_port, # 5432 for PostgreSQL, 5439 for Redshift
      database=db_name,
      user=db_user,
      password=db_password,
   )
   cursor = conn.cursor()

   query = """
      SELECT customer_id, current_score, current_limit, application_date
      FROM batch_scores
      WHERE current_score >= 600
        AND current_limit < 10000
        AND limit_increase_decision IS NULL
      ORDER BY current_score DESC
      LIMIT %s
   """

   cursor.execute(query, (limit,))
   customers = [
      {"customer_id": row[0], "current_score": row[1],
       "current_limit": float(row[2]), "application_date": str(row[3])}
      for row in cursor.fetchall()
   ]

   # Save to JSON file for next pipeline step
   if output_path:
      with open(output_path, "w") as f:
         json.dump(customers, f, indent=2)

   return customers
```

SageMaker Inference Processing Step

This Processing Step runs CatBoost model inference to calculate updated credit scores for each customer using SHAP values. The model is loaded from the container's model directory (`/opt/ml/model`) where it was placed by the training step output. Since SageMaker training outputs models as `model.tar.gz` files, the processing script first extracts the archive (if present), then loads the model using `joblib.load()` from the extracted `catboost_model.joblib` file. The inference process uses SHAP values to compute feature contributions, converts log-odds to credit scores using the PDO (Points to Double Odds) method, and outputs scored customers for the next processing step.

Alternative Deployment Options

This chapter uses direct model loading within Processing Steps. Other deployment approaches will be covered in subsequent chapters:

- **SageMaker Endpoints**: Deploying models to SageMaker endpoints (local mode and production) for inference
- **Docker Lambda**: Using Docker-based Lambda functions for larger models that exceed the 250 MB limit

```
import argparse
import json
import os
import tarfile
import joblib
import pandas as pd
import numpy as np
from catboost import Pool

def parse_args():
   """Parse command line arguments"""
   parser = argparse.ArgumentParser()
   parser.add_argument("--model-dir", type=str, default="/opt/ml/model")
   return parser.parse_args()

def load_model(model_dir):
   """Load CatBoost model and metadata"""
   # Check if model is in a tar.gz file (from SageMaker training)
   tar_files = [f for f in os.listdir(model_dir) if f.endswith(".tar.gz")]
   if tar_files:
      # Extract the tar.gz file
      tar_path = os.path.join(model_dir, tar_files[0])
      with tarfile.open(tar_path, "r:gz") as tar:
         tar.extractall(path=model_dir)

   model_path = os.path.join(model_dir, "catboost_model.joblib")
   metadata_path = os.path.join(model_dir, "model_metadata.json")

   model = joblib.load(model_path)
   with open(metadata_path, "r") as f:
      metadata = json.load(f)

   return model, metadata

def calculate_score(model, metadata, customer_features):
   """Calculate score using CatBoost model with SHAP values"""
   feature_names = metadata["feature_names"]
   categorical_features = metadata.get("categorical_features", [])

   # Create DataFrame with correct feature order
   X_dict = {name: customer_features.get(name, 0) for name in feature_names}
   X = pd.DataFrame([X_dict])

   # Convert categorical features to string
   for cat_feat in categorical_features:
      if cat_feat in X.columns:
         X[cat_feat] = X[cat_feat].astype(str).fillna("NA")

   # Get categorical feature indices
   cat_feature_indices = [
```

```
        feature_names.index(f) for f in categorical_features if f in feature_names
    ]

    # Create CatBoost Pool
    pool = Pool(X, cat_features=cat_feature_indices or None)

    # Get SHAP values
    shap_values = model.get_feature_importance(type="ShapValues", data=pool)

    # SHAP values shape: (n_samples, n_features + 1)
    feature_shap = shap_values[:, :-1] # Feature contributions
    base_shap = shap_values[:, -1] # Base value

    # Sum SHAP values to get total log-odds
    log_odds = feature_shap.sum(axis=1)[0] + base_shap[0]

    # Convert to score using PDO formula
    factor = metadata["factor"]
    offset = metadata["offset"]
    score = offset + factor * (-log_odds)

    return float(score)

def main():
    """Main processing function"""
    args = parse_args()

    # Load model
    model, metadata = load_model(args.model_dir)

    # Load customers from input (SageMaker Processing input directory)
    input_dir = "/opt/ml/processing/input/customers"
    customers_file = os.path.join(input_dir, "customers.json")

    with open(customers_file, "r") as f:
        customers = json.load(f)

    # Score customers
    feature_names = metadata.get("feature_names", [])
    scored_customers = []

    for customer in customers:
        # Generate features (or load from database)
        features = generate_sample_features(customer, feature_names)

        # Calculate score
        new_score = calculate_score(model, metadata, features)

        scored_customers.append({
            "customer_id": customer["customer_id"],
            "current_score": customer["current_score"],
            "current_limit": customer["current_limit"],
            "new_score": new_score,
        })

    # Save to output (SageMaker Processing output directory)
    output_path = "/opt/ml/processing/output/scored_customers.json"
    os.makedirs(os.path.dirname(output_path), exist_ok=True)

    with open(output_path, "w") as f:
        json.dump(scored_customers, f, indent=2)
```

```
if __name__ == "__main__":
   main()
```

Lambda Function for Limit Management

This Lambda function (`limit_manager.py`) applies business rules to determine limit changes (INCREASE, DECREASE, or KEEP) and saves decisions to S3.

```
import json
import os
import boto3

LOCALSTACK_ENDPOINT = os.environ.get("LOCALSTACK_ENDPOINT", "http://localhost:4566")
S3_BUCKET = os.environ.get("S3_BUCKET", "credit-scoring-models")
REGION = os.environ.get("AWS_DEFAULT_REGION", "us-east-1")
S3_ENDPOINT = os.environ.get(
   "S3_ENDPOINT", f"http://s3.{REGION}.localhost.localstack.cloud:4566"
)

s3_client = boto3.client("s3", endpoint_url=S3_ENDPOINT)

def lambda_handler(event, context):
   """Process scored customers and make limit change decisions."""
   # Get S3 location from event (from previous pipeline step)
   s3_uri = event.get("s3_uri")

   # Parse S3 URI and read file
   if s3_uri.startswith("s3://"):
      s3_uri = s3_uri[5:]
   parts = s3_uri.split("/", 1)
   bucket = parts[0]
   key = parts[1] if len(parts) > 1 else ""

   # Read scored customers file from S3
   response = s3_client.get_object(Bucket=bucket, Key=key)
   scored_customers = json.loads(response["Body"].read().decode("utf-8"))

   # Process each customer
   decisions = []
   for customer in scored_customers:
      customer_id = customer.get("customer_id")
      new_score = float(customer.get("new_score", 0))
      current_limit = float(customer.get("current_limit", 0))

      # Business rules for limit increase/decrease
      decision = "KEEP"
      new_limit = current_limit
      reason = "No change"

      if new_score >= 700 and current_limit < 10000:
         # High score, increase limit
         increase_pct = min(50, (new_score - 600) / 2)
         new_limit = int(current_limit * (1 + increase_pct / 100))
         new_limit = min(new_limit, 10000)
         decision = "INCREASE"
```

```
            reason = f"High score ({new_score:.0f}), eligible for increase"
        elif new_score >= 600 and current_limit < 5000:
            # Medium score, moderate increase
            increase_pct = min(25, (new_score - 550) / 2)
            new_limit = int(current_limit * (1 + increase_pct / 100))
            new_limit = min(new_limit, 5000)
            decision = "INCREASE"
            reason = f"Good score ({new_score:.0f}), moderate increase"
        elif new_score < 500:
            # Low score, decrease limit
            decrease_pct = max(10, (500 - new_score) / 10)
            new_limit = int(current_limit * (1 - decrease_pct / 100))
            new_limit = max(new_limit, 1000)
            decision = "DECREASE"
            reason = f"Low score ({new_score:.0f}), reducing limit"

        decisions.append({
            "customer_id": customer_id,
            "current_score": customer.get("current_score"),
            "new_score": new_score,
            "current_limit": current_limit,
            "new_limit": new_limit,
            "decision": decision,
            "reason": reason,
        })

    # Save decisions to S3
    output_key = "pipeline/decisions/decisions.json"
    s3_client.put_object(
        Bucket=S3_BUCKET,
        Key=output_key,
        Body=json.dumps(decisions, indent=2),
        ContentType="application/json",
    )

    return {
        "statusCode": 200,
        "decisions_count": len(decisions),
        "s3_uri": f"s3://{S3_BUCKET}/{output_key}",
    }
```

Update Database Script

This script (`scripts/database.py`) updates Amazon Redshift (or PostgreSQL locally) with limit change decisions from the Lambda function output.

```
import json
import psycopg2
from datetime import datetime
import argparse

def update_database(decisions_file, db_host, db_port, db_name, db_user, db_password):
    """Update database with decisions from JSON file."""
    # Read decisions from file
    with open(decisions_file, "r") as f:
        decisions = json.load(f)
```

```
    # Connect to database
    conn = psycopg2.connect(
        host=db_host,
        port=db_port,
        database=db_name,
        user=db_user,
        password=db_password
    )
    cursor = conn.cursor()

    # Update each customer record
    for decision in decisions:
        update_query = """
            UPDATE batch_scores
            SET limit_increase_decision = %s,
                new_limit = %s,
                decision_reason = %s,
                updated_at = %s
            WHERE customer_id = %s
        """

        cursor.execute(update_query, (
            decision['decision'],
            decision['new_limit'],
            decision['reason'],
            datetime.now(),
            decision['customer_id']
        ))

    conn.commit()
    cursor.close()
    conn.close()

    print(f"Updated {len(decisions)} customer records")

if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("update")
    parser.add_argument("--decisions-file", required=True)
    parser.add_argument("--db-host", required=True)
    parser.add_argument("--db-port", type=int, required=True)
    parser.add_argument("--db-name", required=True)
    parser.add_argument("--db-user", required=True)
    parser.add_argument("--db-password", required=True)
    args = parser.parse_args()

    update_database(
        args.decisions_file,
        args.db_host,
        args.db_port,
        args.db_name,
        args.db_user,
        args.db_password
    )
```

Creating a Processing Step

This example shows how to create a Processing Step that connects to a previous step's output:

```
from sagemaker.workflow.steps import ProcessingStep
from sagemaker.processing import ProcessingInput, ProcessingOutput

def create_inference_step(processor, query_step, training_step):
   """Step 2: Run inference using CatBoost model"""
   script_path = "scripts/inference_processing.py"

   step = ProcessingStep(
      name="SageMakerInference",
      processor=processor,
      inputs=[
         ProcessingInput(
            source=query_step.properties.ProcessingOutputConfig.Outputs[
               "customers"
            ].S3Output.S3Uri,
            destination="/opt/ml/processing/input/customers",
            input_name="customers",
         ),
         ProcessingInput(
            source=training_step.properties.ModelArtifacts.S3ModelArtifacts,
            destination="/opt/ml/model",
            input_name="model",
         ),
      ],
      outputs=[
         ProcessingOutput(
            output_name="scored_customers",
            source="/opt/ml/processing/output",
            destination="s3://credit-scoring-models/pipeline/scored_customers",
         )
      ],
      code=script_path,
      job_arguments=["--model-dir", "/opt/ml/model"],
      depends_on=[query_step, training_step],
   )
   return step
```

Creating the Complete Pipeline

This example shows how to create and register the complete pipeline:

```
from sagemaker.workflow.pipeline import Pipeline
from sagemaker.local import LocalSession
from sagemaker.processing import ScriptProcessor

# Configure LocalSession for local mode
sagemaker_session = LocalSession(boto_session=boto3_session)
sagemaker_session.config = {"local": {"local_code": True}}

DUMMY_IAM_ROLE = (
   "arn:aws:iam::111111111111:role/service-role/"
```

```
    "AmazonSageMaker-ExecutionRole-20200101T000001"
)

# Create shared processor
processor = ScriptProcessor(
    role=DUMMY_IAM_ROLE,
    image_uri="python:3.11-slim",
    command=["python"],
    instance_type="local",
    sagemaker_session=sagemaker_session,
)

# Create steps (in order)
processor = create_processor()
training_step = create_training_step()
query_step = create_query_step(processor)
inference_step = create_inference_step(processor, query_step, training_step)

# Create pipeline
pipeline = Pipeline(
    name="BatchCreditScoringPipeline",
    steps=[training_step, query_step, inference_step],
    sagemaker_session=sagemaker_session,
)

# Register pipeline
pipeline.upsert(role_arn=DUMMY_IAM_ROLE)

# Execute pipeline
execution = pipeline.start()
execution.wait()

# After pipeline execution completes:
# 1. Invoke Lambda function to make decisions
scored_customers_uri = f"s3://{S3_BUCKET}/pipeline/scored_customers/scored_customers.
    json"
decisions_uri = invoke_lambda_function(scored_customers_uri)

# 2. Update database with decisions
update_database_from_s3(decisions_uri)

# Register pipeline
pipeline.upsert(role_arn=DUMMY_IAM_ROLE)
```

Executing SageMaker Pipelines

This example shows how to execute a SageMaker pipeline (can be triggered manually or by EventBridge on schedule).

```

# Execute pipeline
execution = pipeline.start()

# Handle local mode (no arn attribute)
execution_id = (
    getattr(execution, "arn", None)
    or getattr(execution, "execution_arn", None)
```

```
    or "local-execution"
)
print(f"Pipeline execution started: {execution_id}")

# Wait for completion (local mode completes synchronously)
try:
    execution.wait()
except AttributeError:
    # Local mode - execution completes synchronously
    print("Execution completed (local mode)")

# Get execution status
try:
    execution_status = execution.describe()["PipelineExecutionStatus"]
except (AttributeError, KeyError, TypeError):
    # Local mode might have different structure
    execution_status = getattr(execution, "status", "Succeeded")

print(f"Status: {execution_status}")
```

Querying Results from Redshift

This example shows how to query the updated limit change decisions from Amazon Redshift (or PostgreSQL locally).

```
# scripts/database.py - check command
import psycopg2

# Connect to Redshift (or PostgreSQL locally)
# Redshift uses port 5439, PostgreSQL uses port 5432
conn = psycopg2.connect(
    host='localhost',
    port=5432, # 5432 for PostgreSQL, 5439 for Redshift
    database='credit_scoring',
    user='creditrisk',
    password='creditrisk123'
)
cursor = conn.cursor()

# Get summary of decisions
cursor.execute("""
    SELECT
        COUNT(*) as total,
        COUNT(CASE WHEN limit_increase_decision = 'INCREASE' THEN 1 END) as increase,
        COUNT(CASE WHEN limit_increase_decision = 'DECREASE' THEN 1 END) as decrease,
        COUNT(CASE WHEN limit_increase_decision = 'NO_CHANGE' THEN 1 END) as no_change
    FROM batch_scores
""")

summary = cursor.fetchone()
print(f"Total: {summary[0]}, Increase: {summary[1]}, "
      f"Decrease: {summary[2]}, No change: {summary[3]}")

# Get customers with limit increases
cursor.execute("""
    SELECT customer_id, current_score, current_limit, new_limit, decision_reason
```

```
    FROM batch_scores
    WHERE limit_increase_decision = 'INCREASE'
    ORDER BY new_limit DESC
    LIMIT 10
""")

for row in cursor.fetchall():
    customer_id, score, old_limit, new_limit, reason = row
    print(f"{customer_id}: ${old_limit:,.0f} -> ${new_limit:,.0f} (Score: {score})")
```

Key Takeaways

What You've Learned

- **EventBridge**: Schedule automated batch processing jobs via Lambda functions
- **SageMaker Pipelines**: Orchestrate ML workflows with:
  - Processing Steps for training and inference
  - Versioning and tracking capabilities
  - CatBoost gradient boosting for credit scoring
  - SHAP values for feature importance and scoring
- **Lambda Functions and Business Rules**: Integrate limit management logic (INCREASE/DECREASE/KEEP) with clear eligibility criteria
- **Amazon Redshift**: Store and query batch credit scores efficiently (PostgreSQL-compatible for local development)
- **Batch Processing**: Process existing customers in scheduled batches vs. real-time applications

Further Reading

Additional Resources

For more information about SageMaker Pipelines and related topics:

- **GitHub Repository**: github.com/deburky/credit-risk-modeling-on-aws
- **Amazon SageMaker Pipelines**: aws.amazon.com/sagemaker/ai/pipelines
- **SageMaker MLflow Pipelines**: medium.com/aws-in-plain-english (covers MLflow integration with SageMaker Pipelines)